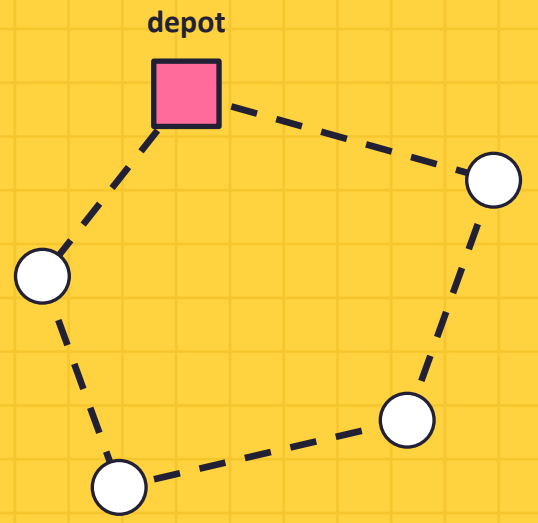
DAA Mini Project · Topic 3

Smart Delivery Route Planning System



Finding the shortest delivery route with the Travelling Salesman Problem, using Greedy, Dynamic Programming and Branch & Bound

Course: Design and Analysis of Algorithms

Semester: 5th

Faculty: Yamini Dubey

Team

Palak	Enrollment 12402080601100
Pearl	Enrollment12402080601104
Prince	Enrollment 12402080601107
Robin	Enrollment 12402080601020

Introduction

Problem statement

A delivery van leaves the depot, has to visit every delivery address exactly once, and must come back. In which order should it go so the total distance is as small as possible?

This is the Travelling Salesman Problem. It is NP-hard: with n stops there are (n − 1)! possible routes.

Our system computes the route with each algorithm, draws it on a map and reports distance, time and how far each answer is from the best one.

Input	n locations as (x, y) km or latitude/longitude; location 0 is the depot
Output	A closed route 0 → … → 0, its total distance (km) and run time (ms)
Constraints	Every stop visited exactly once; start and end at the depot; distances ≥ 0
Objective	Minimise the sum of dist(route[k], route[k+1])

Introduction

Why delivery routes matter

Cost

- Fuel and driver hours are the biggest last-mile costs
- A route 10–15% shorter saves that much fuel on every trip
- More deliveries per van per shift

Where it's used

- Courier and e-commerce delivery
- Food and grocery apps
- Garbage collection and school buses
- Warehouse picking and PCB drilling

Who benefits

- Companies: lower running cost
- Drivers: fewer km and hours
- Customers: faster, predictable delivery
- Cities: less traffic and pollution

Introduction

Objectives

Model

Represent stops as a complete weighted graph (n × n distance matrix)

Implement

Brute Force, Greedy Nearest Neighbour, Held-Karp DP and LC Branch & Bound

Verify

Check DP and B&B give the brute-force optimum on 30+ random inputs

Measure

Record run time from n = 5 to 2000 stops, 5 random inputs per size

Analyse

Best, average and worst-case time and space; compare with measured growth

Show it

Website that draws each route on a map and compares the four methods

Background

The obvious way: try every route

Fix the depot, list every order of the other n − 1 stops, add up each route and keep the shortest. It is always correct, but the work explodes.

Limitations

- Time grows as O(n!): one extra stop multiplies the work
- We measured 183 ms at n = 10; past ~12 stops it is useless
- Planning by hand or in a fixed order ignores distance
- A real delivery van often has dozens of stops

So we need methods that either find the optimum much faster (DP, Branch & Bound) or give a good route instantly for big inputs (Greedy).

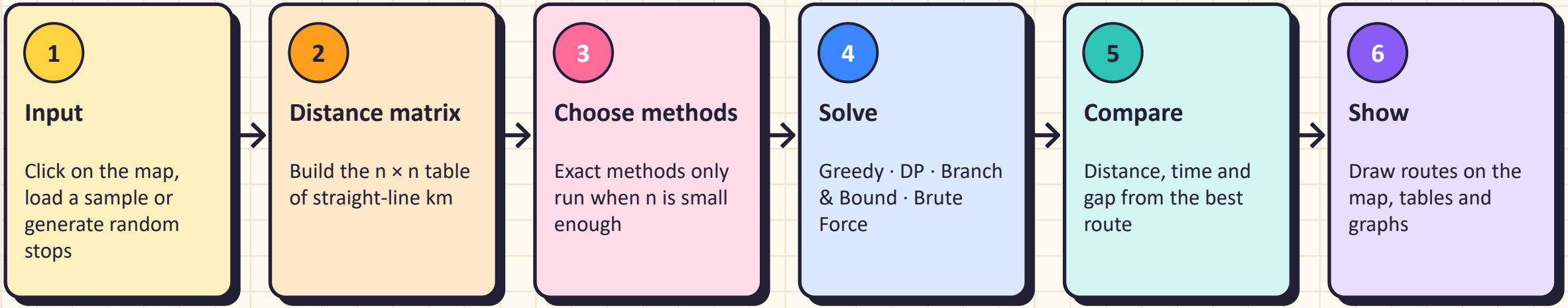
Routes to check = (n − 1)!

Stops (n)	Possible routes
5	**24**
10	**362,880**
15	**87,178,291,200**
20	**about 1.2×10^{17}**

At a billion routes per second, 20 stops would take about 3.9 years.

Design

Our approach



Why each step

- The distance matrix is built once and shared, so every algorithm solves exactly the same input
- Greedy always runs ($O(n^2)$); DP runs up to 18 stops and Branch & Bound up to 14, so the website never freezes
- The best exact answer is the yardstick: the gap shows how much distance greedy gives away

Design

Algorithms used

Greedy · Nearest Neighbour

Idea: From where you are, drive to the closest stop not yet visited.

Why: Fastest; handles thousands of stops; gives B&B its first upper bound

Data structures: visited[] array

Dynamic Programming · Held-Karp

Idea: dp[S][j] = shortest path from the depot through set S ending at j; reuse those answers.

Why: Exact and far faster than n!: $O(n^2 \cdot 2^n)$

Data structures: $2^n \times n$ table, bitmask for S, parent table

Branch & Bound · LC search

Idea: Grow partial routes cheapest-bound-first; cut any branch whose bound ≥ best route so far.

Why: Exact; cuts most of the search tree on normal inputs

Data structures: Min-heap, reduced cost matrix per node

Brute Force (baseline)

Idea: Try all (n − 1)! orders and keep the shortest.

Why: Ground truth to check the other three

Data structures: Permutation generator

Design

Pseudocode

Held-Karp (DP)

```
dp[{0}][0] ← 0, all others ← ∞
for each subset S that contains 0:
  for each j in S, j ≠ 0:
    dp[S][j] ← min over i in S−{j}
       of dp[S−{j}][i] + d[i][j]
       (remember parent[S][j] ← i)
cost ← min over j of
       dp[ALL][j] + d[j][0]
walk parent[][] back to get route
```

Time Θ(n²·2ⁿ) · Space Θ(n·2ⁿ)

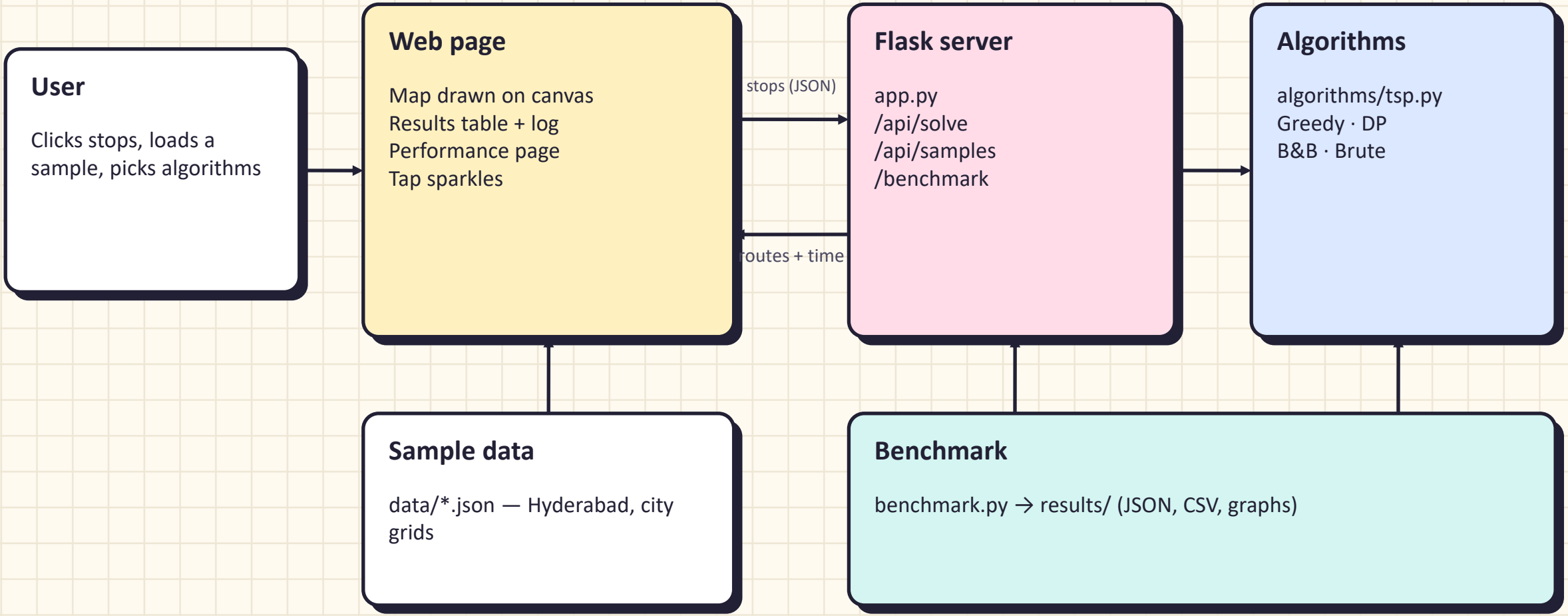
LC Branch & Bound

```
M ← reduce(d); root.bound ← reduction
best ← NearestNeighbour(d).cost
push root into min-heap
while heap not empty:
  node ← pop smallest bound
  if node.bound ≥ best: prune
  if route complete: update best
  for each unvisited j:
    child ← M without row i, col j
    bound ← node.bound + M[i][j]
            + reduce(child)
    if bound < best: push child
```

Time O(n²·n!) worst, ~O(n³) best · Space O(n² × live nodes)

Implementation

How the system fits together



Implementation

Implementation and who did what

Built with

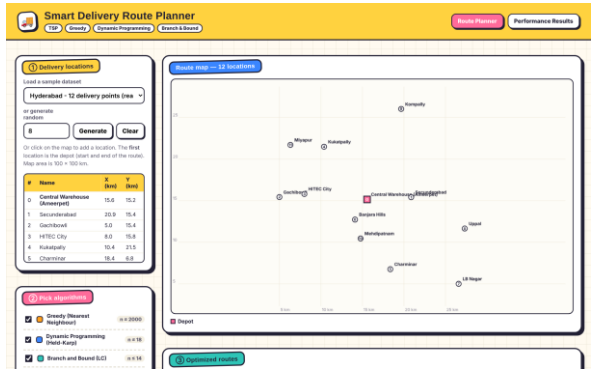
- Python 3 — algorithms and server
- Flask — web backend
- HTML, CSS, JavaScript canvas — map and sparkles
- Matplotlib — graphs
- No TSP library: all four algorithms written by us

All four tested the website together and prepared the final demo.

Member	Part of the project	What they did
Palak	**Problem study and Greedy**	Problem analysis and background study; Greedy Nearest Neighbour (nearest_neighbour); sample datasets incl. Hyderabad stops; report writing
Pearl	**Dynamic Programming and testing**	Held-Karp DP with bitmask table and route rebuild (held_karp); complexity analysis; unit tests checking DP and B&B against brute force
Prince	**Branch & Bound and benchmarks**	LC Branch & Bound with reduced cost matrix (branch_and_bound); Brute Force baseline; benchmark.py, result tables and graphs
Robin	**Website and integration**	Flask backend (app.py, /api/solve); route map, results page and tap sparkles; connecting all parts; screenshots and presentation

Results

The project running



Input: 12 real Hyderabad stops loaded



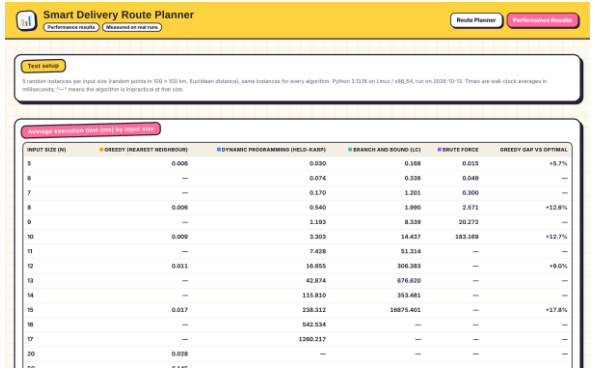
Output: greedy (orange) vs optimal routes

③ Optimized routes

Greedy (Nearest Neighbour)	Dynamic Programming (Held-Karp)	Branch and Bound (LC)
77.73 km	75.03 km	75.03 km
0.02 ms	28.705 ms	32.044 ms

ALGORITHM	DISTANCE (KM)	TIME (MS)	GAP VS BEST	TYPE	DELIVERY ORDER
Greedy (Nearest Neighbour)	77.735	0.02	+3.61%	Heuristic	Central Warehouse (Ameerpet) → Banjara Hills → Mehdipatnam → Charminar → LB Nagar → Uppal → Secunderabad → Kompally → Kukatpally → Miyapur → HITEC City → Gachibowli → Central Warehouse (Ameerpet)
Dynamic Programming (Held-Karp)	75.025	28.705	best	Exact (optimal)	Central Warehouse (Ameerpet) → Banjara Hills → Mehdipatnam → Charminar → LB Nagar → Uppal → Secunderabad → Kompally → Kukatpally → Miyapur → Gachibowli → HITEC City → Central Warehouse (Ameerpet)
Branch and Bound (LC)	75.025	32.044	best	Exact (optimal)	Central Warehouse (Ameerpet) → Banjara Hills → Mehdipatnam → Charminar → LB Nagar → Uppal → Secunderabad → Kompally → Kukatpally → Miyapur → Gachibowli → HITEC City → Central Warehouse (Ameerpet)

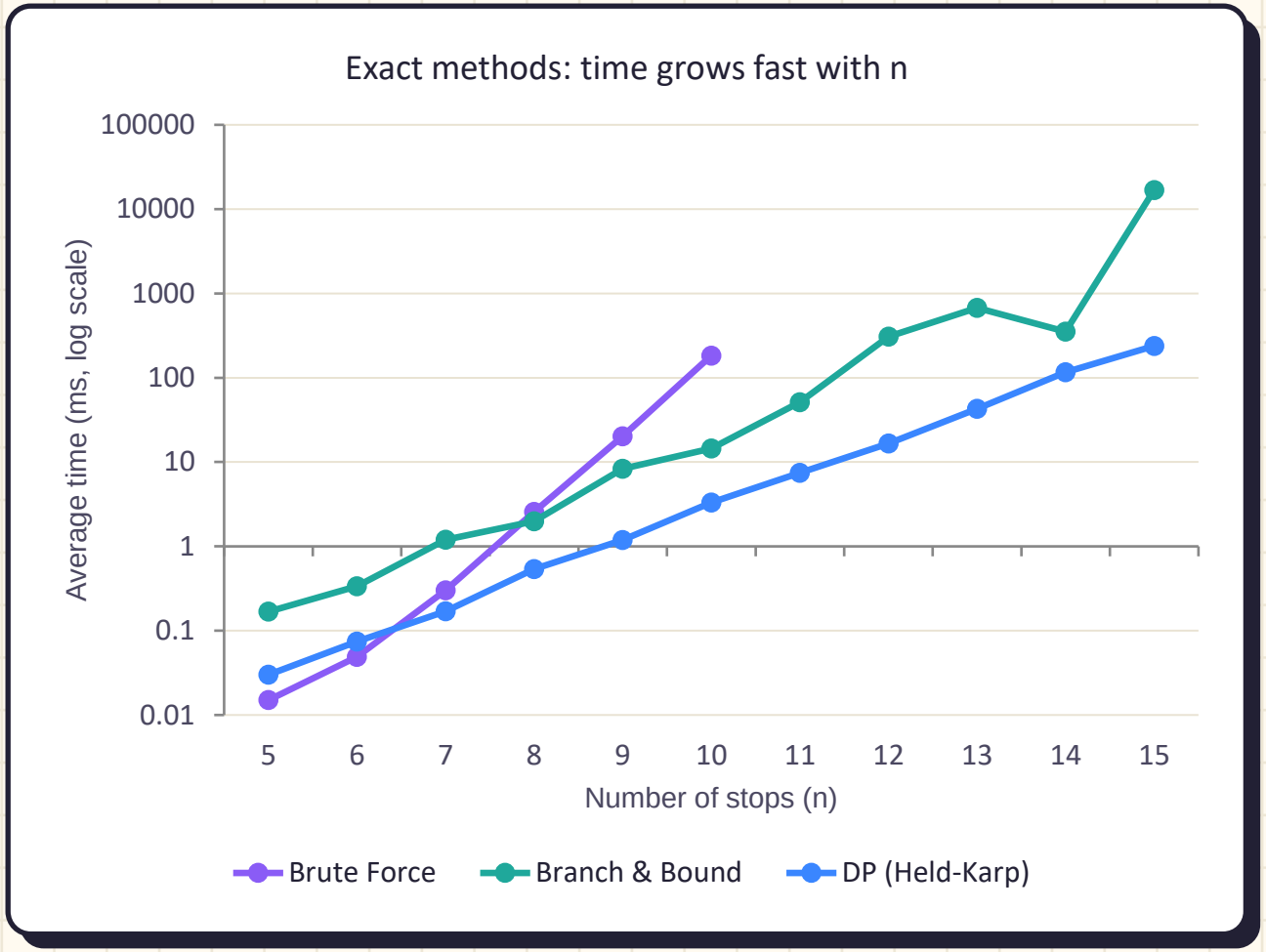
Distance, time and gap for each algorithm



Performance page with measured times

Results

Results



n	Greedy	DP	B&B	Brute
5	0.006 ms	0.030 ms	0.168 ms	0.015 ms
10	0.009 ms	3.303 ms	14.4 ms	183.2 ms
15	0.017 ms	238.3 ms	16.88 s	—
50	0.145 ms	—	—	—
100	0.543 ms	—	—	—
500	20.3 ms	—	—	—
2000	405.9 ms	—	—	—

Average of 5 random inputs per size. "—" = not run (too slow at that size).

Greedy is 5.7–17.8% longer than the best route, but solves 2000 stops in 0.41 s.

Hyderabad, 12 stops: greedy 77.73 km, optimal 75.03 km (+3.6%).

Results

Complexity analysis

Algorithm	Best case	Average case	Worst case	Space
Greedy (NN)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$
DP (Held-Karp)	$\Theta(n^2 \cdot 2^n)$	$\Theta(n^2 \cdot 2^n)$	$\Theta(n^2 \cdot 2^n)$	$\Theta(n \cdot 2^n)$
Branch & Bound	$\approx O(n^3)$	exponential, depends on input	$O(n^2 \cdot n!)$	$O(n^2 \times$ live nodes$)$
Brute Force	$\Theta(n \cdot n!)$	$\Theta(n \cdot n!)$	$\Theta(n \cdot n!)$	$\Theta(n)$

Theory vs what we measured

- **Greedy:** doubling n (500 → 1000 → 2000) made it 4.3–4.7× slower, as n^2 predicts
- **DP:** each extra stop made it 2.1–2.7× slower; the curve sits on $c \cdot n^2 \cdot 2^n$ for n = 5–17
- **B&B:** at n = 15 the fastest input took 0.39 s and the slowest 74.5 s; it explored about 44,000 nodes, not 8.7×10^{10} routes
- **Brute:** each extra stop made it 7.9–9× slower, matching $(n - 1)!$

Conclusion

Strengths, limits and what's next

Strengths

- Exact best route for up to 15 to 18 stops
- Instant good routes for thousands of stops
- All methods solve the same input, so the comparison is fair
- Works with real latitude/longitude
- Every method checked against brute force

Limitations

- Straight-line distance, not road distance
- Exact methods are exponential
- B&B time is hard to predict
- One van; no time windows or capacity
- Python is slower than C++

Future scope

- Road distances from a maps API
- Improve greedy routes with the 2-opt method
- Several vans with capacity (VRP)
- Delivery time windows and priorities
- Genetic or ant-colony methods for 1000+ stops

Conclusion

Conclusion and references

- DP and Branch & Bound always matched the brute-force best route
- Held-Karp was the steadiest exact method: 1.26 s for 17 stops
- B&B skipped more than 99.9% of routes, but its time changed 190× between inputs of the same size
- Greedy is the only choice for big inputs: 2000 stops in 0.41 s, for about 6–18% extra distance
- Pick by size: exact methods for small routes, greedy (plus improvement) for large ones

References

1. E. Horowitz, S. Sahni, S. Rajasekaran — Fundamentals of Computer Algorithms, 2nd ed.
2. T. H. Cormen et al. — Introduction to Algorithms, 3rd ed., MIT Press
3. M. Held, R. M. Karp — “A Dynamic Programming Approach to Sequencing Problems”, J. SIAM, 1962
4. D. Applegate et al. — The Traveling Salesman Problem: A Computational Study, Princeton, 2006
5. Flask documentation — flask.palletsprojects.com
6. Matplotlib documentation — matplotlib.org